Imports

In [514…
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import base64
```

In [ ]:
```
```

In [515…
```
import ratinabox
from ratinabox.Environment import Environment
from ratinabox.Agent import Agent
from ratinabox.Neurons import FieldOfViewOVCs
# from ratinabox.Neurons import PlaceCells, GridCells #...
```

aux funcs

In [516…
```python
import random
import string

def generate_random_string(n):
    """
    Generate a random string of length n.

    Parameters:
    n (int): The desired length of the string.

    Returns:
    str: A random string of length n.
    """
    if n <= 0:
        raise ValueError("Length 'n' must be a positive integer.")

    # Combine letters, digits, and punctuation
    characters = string.ascii_uppercase

    # Generate a random string of length n
    random_string = ''.join(random.choices(characters, k=n))

    return random_string
```

In [517…
```python
generate_random_string(8)
```

Out[517… 'ECPPARSF'

In [518…
```python
def get_rand_letter(x):
    if x <= 0.2:
        return 'A'
    elif x > 0.2 and x <= 0.4:
        return 'B'
    elif x > 0.4 and x <= 0.6:
        return 'C'
    elif x > 0.6 and x <= 0.8:
        return 'D'
    else:
        return 'E'
```

In [519…
```python
def is_point_in_hole(point, hole):
    """
    Verifica se um ponto está dentro de um polígono (hole).
    """
    x, y = point
```

```
    n = len(hole)
    inside = False

    px, py = hole[0]
    for i in range(1, n + 1):
        cx, cy = hole[i % n]
        if y > min(py, cy):
            if y <= max(py, cy):
                if x <= max(px, cx):
                    if py != cy:
                        xinters = (y - py) * (cx - px) / (cy - py) + px
                        if px == cx or x <= xinters:
                            inside = not inside
        px, py = cx, cy

    return inside
```

In [520…

```
def is_point_valid(point, hole):
    x_valid = True
    y_valid = True
    point_valid = True
    hole_x_inf = hole[0][0]
    hole_x_sup = hole[1][0]
    hole_y_inf = hole[0][1]
    hole_y_sup = hole[1][1]
    # check x pos
    if (point[0] > hole_x_inf) and (point[0] < hole_x_sup):
        x_valid = False
    if (point[1] > hole_y_inf) and (point[1] < hole_y_sup):
        y_valid = False
    if (not x_valid or  not y_valid) or (not x_valid and y_valid):
        point_valid = False
    # print(point, point_valid, x_valid, y_valid)
    return point_valid
```

In [521…

```
def get_euclidean_distance(pos1, pos2):
    return np.sqrt((pos1[0] - pos2[0])**2 + ((pos1[1] - pos2[1])**2))
def apply_eucd_dist(x, obj=9):
    return get_euclidean_distance(x['pos'], x[f'obj_{obj}_pos'])
```

In [522…

```
hole_deff = [[0.4,0.2],[0.6,0.2],[0.6,0.8],[0.4,0.8]]
```

In [523…

```
hole_deff[1][0]
```

Out[523… 0.6

Define EnV

In [524…
```
Env = Environment(params={
    'aspect':1,
    'holes' : [[[0.4,0.2],[0.6,0.2],[0.6,0.8],[0.4,0.8]]],
    })
```

Num of Cues

In [525…
```
NUM_OF_CUES = 100
```

RUN PARAMETERS

In [526…
```
CUE_NAME_LEN = 4

MAX_NUMBER_OF_CUES = 7
CONTEXT_LEN = MAX_NUMBER_OF_CUES*CUE_NAME_LEN
CONTEXT_LEN
```

Out[526… 28

gerando cues positions

In [527…
```
CUES_LIST = []
num_test = 0
for i in range(NUM_OF_CUES):
    candidate_point = 0.45, 0.55
    while not is_point_valid(candidate_point, hole_deff):
        candidate_point = np.random.rand(), np.random.rand()
        num_test+=1
    CUES_LIST.append(candidate_point)
print(f'{NUM_OF_CUES} cues criadas com {num_test} candidatos')
```

```
100 cues criadas com 127 candidatos
```

adicionando cues ao EnV

In [528…
```
for point in CUES_LIST:
    Env.add_object(object=point,type=0)
```

In [529…
```
len(Env.objects['objects']), NUM_OF_CUES
```

Out[529… (100, 100)

adding name to cue

In [463…

In [530…
```
objects_name_list = []
for i in range(NUM_OF_CUES):
    obj_name = generate_random_string(CUE_NAME_LEN)
    if obj_name in objects_name_list:
        obj_name = generate_random_string(CUE_NAME_LEN)
    objects_name_list.append(obj_name)
```

In [531…
```
len(objects_name_list), len(set(objects_name_list))
```

Out[531… (100, 100)

In [532…
```
# # objetos a direita do buraco
# Env.add_object(object=[0.9,0.3],type=0)
# Env.add_object(object=[0.7,0.1],type=0)
# Env.add_object(object=[0.7,0.5],type=0)
# Env.add_object(object=[0.9,0.7],type=0)
# Env.add_object(object=[0.7,0.9],type=0)


# # objetos a esquerda do buraco
# Env.add_object(object=[0.3,0.3],type=0)
# Env.add_object(object=[0.1,0.1],type=0)
# Env.add_object(object=[0.1,0.5],type=0)
# Env.add_object(object=[0.3,0.7],type=0)
# Env.add_object(object=[0.1,0.9],type=0)
```

In [533…
```
objects_name_list[:3]
```

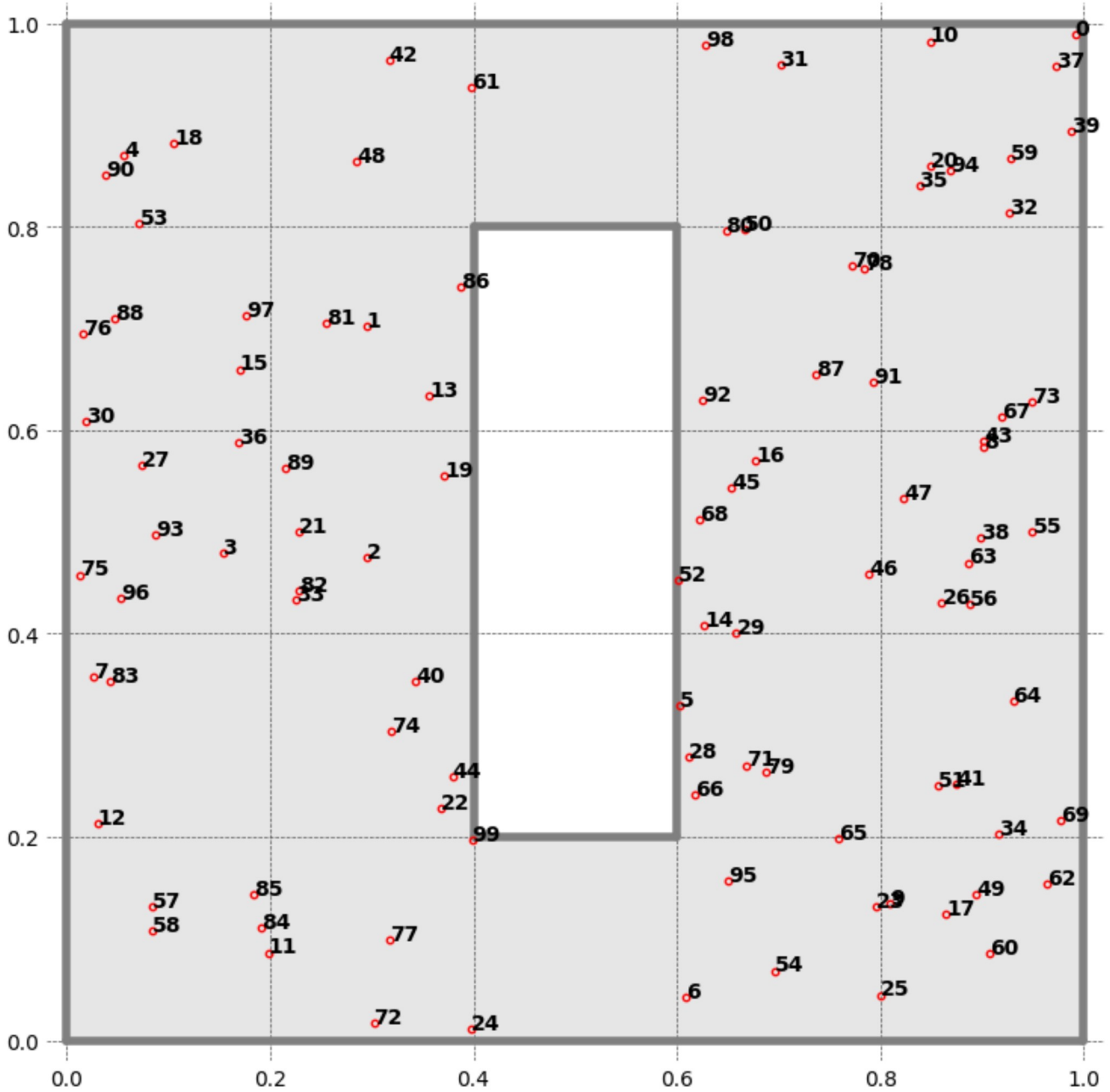
Out[533… ['YRGP', 'SKMP', 'BOIU']

In [534…
```
Env.objects['object_names'] = objects_name_list
```

In [535…
```
# Env.objects
```

vizz env

In [536…
```
fig, ax = plt.subplots(1, 1, figsize=(14,7))
for i in range(NUM_OF_CUES):
    plt.annotate(text=i, xy=Env.objects['objects'][i],fontweight='bold', fontsize=10)
Env.plot_environment(gridlines=True, fig=fig, ax=ax) #visualises current environment with walls and objects
# Agent.plot_trajectory() #plots trajectory
# Agent.animate_trajectory() #animate trajectory
# Neurons.plot_rate_map() # plots the rate map of the neurons at all positions
# Neurons.plot_rate_timeseries() # plots activities of the neurons over time
# Neurons.animate_rate_timeseries() # animates the activity of the neurons over time
plt.show()
```



In [537…
```
Ag = Agent(Env)
```

```
# agent configs
# Ag.pos = start_pos
Ag.thigmotaxis = 0.2
```

In [538…

```
# default_params = {
#         "distance_range": [0.02, 0.4],  # min and max distances the agent can "see"
#         "angle_range": [0,75],  # angluar FoV in degrees (will be symmetric on both sides, so give range in 0 (forwards) to 180 (backwards)
#         "spatial_resolution": 0.02,  # resolution of each BVC tiling FoV
#         "beta": 5, # smaller means larger rate of increase of cell size with radius in hartley type manifolds
#         "cell_arrangement": "diverging_manifold",  # whether all cells have "uniform" receptive field sizes or they grow ("hartley") with radi
#         "object_tuning_type" : None, #can be "random", any integer, or a list of integers of length n. The tuning types of the OVCs.
#     }
```

In [ ]:

In [539…

```
# new_params = {
#         "distance_range": [0.001, 1],  # min and max distances the agent can "see"
#         "angle_range": [0,180],  # angluar FoV in degrees (will be symmetric on both sides, so give range in 0 (forwards) to 180 (backwards)
#         "spatial_resolution": 0.02,  # resolution of each BVC tiling FoV
#         "beta": 5, # smaller means larger rate of increase of cell size with radius in hartley type manifolds
#         "cell_arrangement": "diverging_manifold",  # whether all cells have "uniform" receptive field sizes or they grow ("hartley") with radi
#         "object_tuning_type" : 0, #can be "random", any integer, or a list of integers of length n. The tuning types of the OVCs.
#     }
```

In [540…

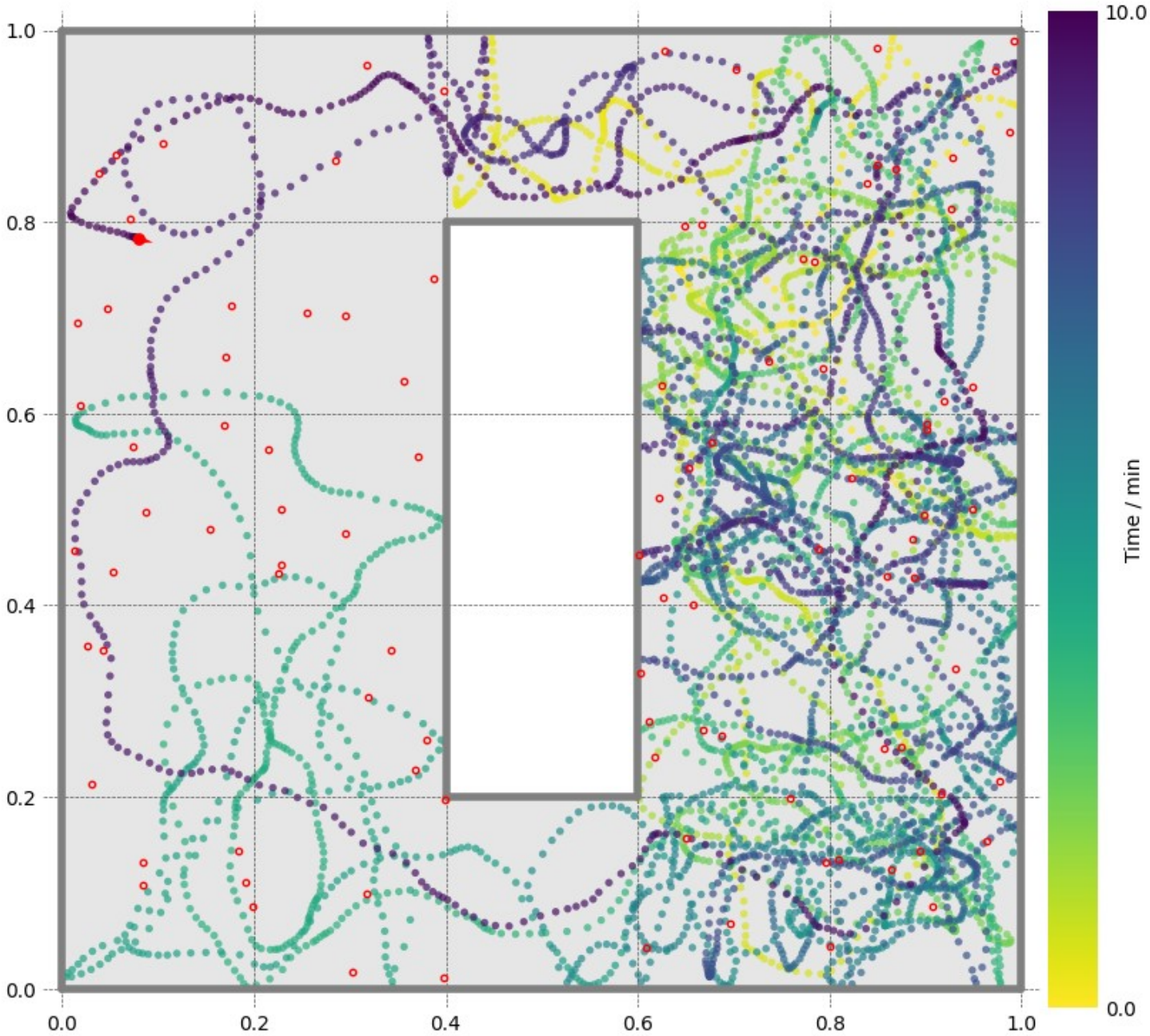
```
# FoV_OVCs = FieldOfViewOVCs(Ag,new_params)
```

In [541…

```
for i in range(int(10*60/Ag.dt)):
    Ag.update()
    # FoV_OVCs.update()
```

In [542…

```
fig, ax = plt.subplots(1, 1, figsize=(14,7))
ax = Ag.plot_trajectory(color='changing', colorbar=True, gridlines=True,fig=fig, ax=ax)
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
10.0
Time / min
0.0

In [543…
```
dfbase = pd.DataFrame(Ag.history).copy()
print(dfbase.shape)
dfbase.head(2)
```

```
(12000, 6)
```

Out[543…

	t	pos	distance_travelled	vel	rot_vel	head_direction
0	0.05	[0.8927268884855031, 0.505292944589053]	0.003808	[-0.02653824673308769, 0.07139564599352832]	0.524318	[-0.33198065972084784, 0.9432861928234243]
1	0.10	[0.8914034882504055, 0.508183352142841]	0.006987	[-0.026468004701951653, 0.057808151075759984]	1.469764	[-0.36041794280043404, 0.932790923255315]

In [544…
```
# df = dfbase.iloc[:150].copy() #dfbase[dfbase.index%10 == 0].copy()
df = dfbase.copy()
df.shape
```

Out[544… (12000, 6)

In [545…
```
# Env.objects
```

In [546…
```
# fixed_object = 8
# Env.objects['objects'][fixed_object]
```

In [547…
```
# Env.objects
```

In [334…

In [548…
```
for i in range(NUM_OF_CUES):
    cue_dict = {f'obj_{i}_pos':[Env.objects['objects'][i] for _ in df.index],
         f'obj_{i}_name':[Env.objects['object_names'][i] for _ in df.index]}
    cue_df = pd.DataFrame(data=cue_dict, index=df.index)
    df = pd.concat([df, cue_df], axis=1)
    # df[f'obj_{i}_pos'] = [Env.objects['objects'][i] for _ in df.index]
    # df[f'obj_{i}_name'] = [Env.objects['object_names'][i] for _ in df.index]
```

In [549…
```
df.shape
```

Out[549… (12000, 206)

In [550…
```
df.head(2)
```

Out[550…

	t	pos	distance_travelled	vel	rot_vel	head_direction	obj_0_pos	obj_0_name	obj_
0	0.05	[0.8927268884855031, 0.505292944589053]	0.003808	[-0.02653824673308769, 0.07139564599352832]	0.524318	[-0.33198065972084784, 0.9432861928234243]	[0.9916465162476297, 0.9887570398068641]	YRGP	[0.294895239852 0.70267500436
1	0.10	[0.8914034882504055, 0.508183352142841]	0.006987	[-0.026468004701951653, 0.057808151075759984]	1.469764	[-0.36041794280043404, 0.932790923255315]	[0.9916465162476297, 0.9887570398068641]	YRGP	[0.294895239852 0.70267500436

2 rows × 206 columns

In [551…
```
for i in range(NUM_OF_CUES):
    dist_cue_dict = {f'dist_to_obj_{i}': df.apply(apply_eucd_dist, axis=1, obj=i)}
    dist_cue_df = pd.DataFrame(data=dist_cue_dict, index=df.index)
    df = pd.concat([df, dist_cue_df], axis=1)
```

In [552…
```
f"{100*len(df)} iteracoes"
```

Out[552… '1200000 iteracoes'

In [553…
```
df.shape
```

Out[553… (12000, 306)

In [554…
```
df.tail(2)
```

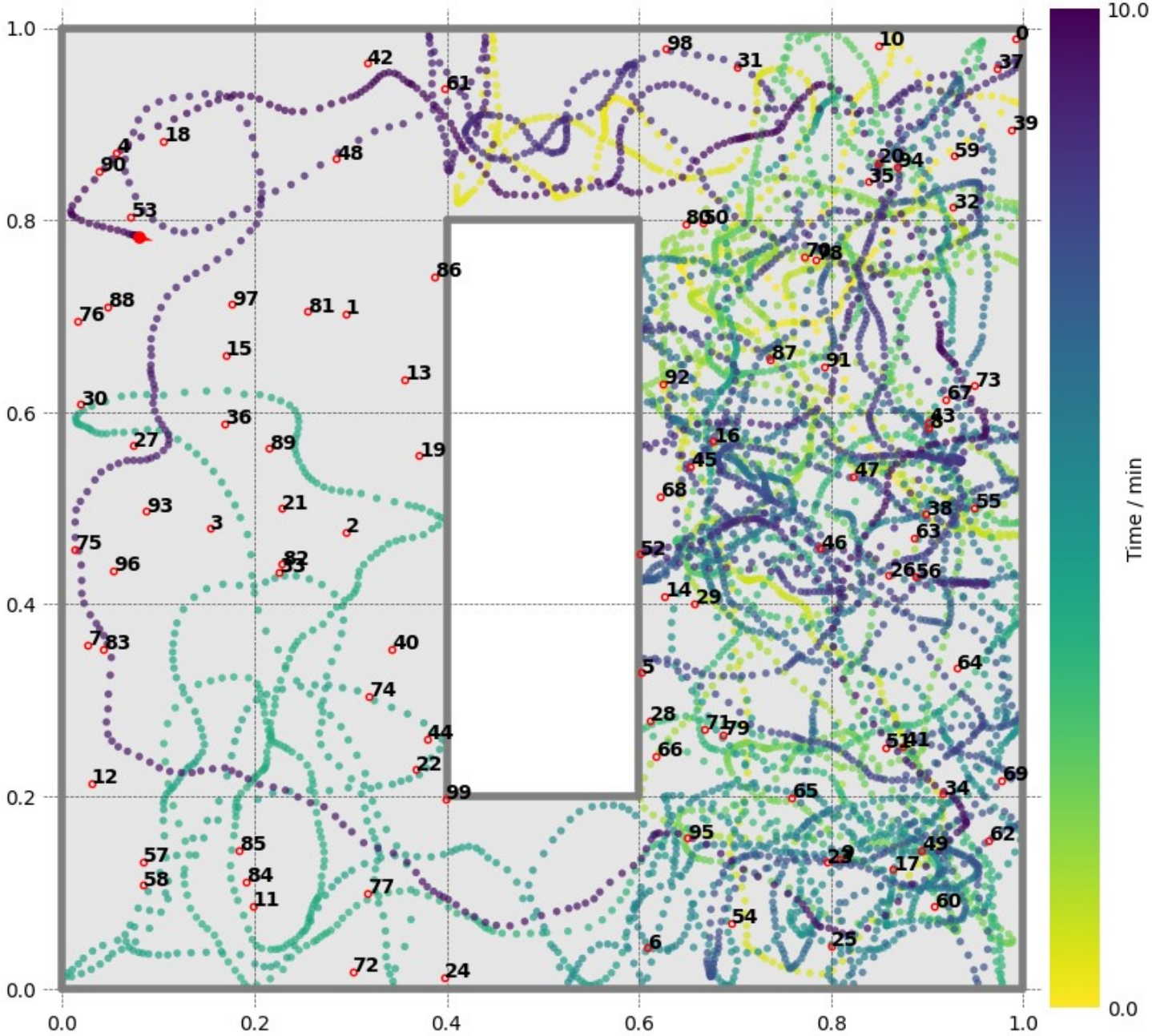
Out[554…

	t	pos	distance_travelled	vel	rot_vel	head_direction	obj_0_pos	obj_0_name	
11998	599.95	[0.0796165352157249, 0.7829671560004151]	58.301326	[0.03447861473355712, -0.010622255638612632]	-0.468570	[0.9757673408398776, -0.21881064085248295]	[0.9916465162476297, 0.9887570398068641]	YRGP	[0.29489 0.7026
11999	600.00	[0.08105514740390966, 0.7824000141500093]	58.302873	[0.028772243763695304, -0.011342837008117002]	-1.533241	[0.9631841000020578, -0.26884268541886347]	[0.9916465162476297, 0.9887570398068641]	YRGP	[0.29489 0.7026

2 rows × 306 columns

In [555…
```
fig, ax = plt.subplots(1, 1, figsize=(14,7))
for i in range(NUM_OF_CUES):
    plt.annotate(text=i, xy=Env.objects['objects'][i], fontsize=10, fontweight='bold')
ax = Ag.plot_trajectory(color='changing', colorbar=True, gridlines=True,fig=fig, ax=ax, t_start=0, t_end=df['t'].max())

plt.show()
```


Time / min
10.0
0.0
0.0
0.2
0.4
0.6
0.8
1.0

In [556…
```
obj_names = [Env.objects['object_names'][i] for i in range(NUM_OF_CUES)]
obj_dist_cols = [f'dist_to_obj_{i}' for i in range(NUM_OF_CUES)]
# def get_position_text(x):
```

In [557…
```
dft = df[obj_dist_cols].T
dft.columns = [f't_{i:08d}' for i in range(df.T.shape[1])]
```

In [ ]:
```
```

In [558…
```
dft.columns[0]
```

Out[558… 't_00000000'

In [559…
```
order = [int(val.split('_')[-1]) for val in dft.sort_values(dft.columns[0]).index[:MAX_NUMBER_OF_CUES]]
dft.sort_values(dft.columns[0])
```

Out[559…

	t_00000000	t_00000001	t_00000002	t_00000003	t_00000004	t_00000005	t_00000006	t_00000007	t_00000008	t_00000009	…	t_00011990
dist_to_obj_38	0.012627	0.015804	0.019733	0.022255	0.024066	0.026551	0.029139	0.031959	0.035894	0.039132	…	0.873591
dist_to_obj_63	0.036174	0.038852	0.041884	0.043766	0.045030	0.046668	0.048471	0.050351	0.053248	0.056017	…	0.871175
dist_to_obj_55	0.056986	0.058621	0.061393	0.063438	0.065103	0.067595	0.070139	0.073099	0.077009	0.079863	…	0.919912
dist_to_obj_47	0.074851	0.072572	0.069336	0.067119	0.065384	0.062839	0.060269	0.057310	0.053450	0.050738	…	0.789755
dist_to_obj_56	0.076810	0.079615	0.082767	0.084687	0.085956	0.087562	0.089291	0.091032	0.093694	0.096291	…	0.887197
…	…	…	…	…	…	…	…	…	…	…	…	…
dist_to_obj_76	0.896749	0.894848	0.891937	0.889874	0.888218	0.885752	0.883253	0.880343	0.876536	0.873838	…	0.106753
dist_to_obj_58	0.900725	0.900819	0.900214	0.899565	0.898858	0.897599	0.896360	0.894704	0.892735	0.891780	…	0.676415
dist_to_obj_12	0.910136	0.909814	0.908701	0.907731	0.906798	0.905238	0.903687	0.901705	0.899275	0.897917	…	0.571978
dist_to_obj_4	0.912648	0.910282	0.906895	0.904566	0.902755	0.900125	0.897450	0.894403	0.890357	0.887348	…	0.087810
dist_to_obj_90	0.921528	0.919220	0.915890	0.913591	0.911797	0.909183	0.906526	0.903490	0.899467	0.896492	…	0.075843

100 rows × 12000 columns

In [560…
```
order
```

Out[560… [38, 63, 55, 47, 56, 8, 26]

In [561…
```
pos_text = ''.join([obj_names[i] for i in order])
pos_text
```

Out[561…
```
'DJEGMTCHVGQKFEIYPCMUBNODHZJA'
```

In [562…
```
# Env.objects['object_names']
```

In [563…
```
positions_list = []
for feat in dft.columns:
    order = [int(val.split('_')[-1]) for val in dft.sort_values(feat).index[:MAX_NUMBER_OF_CUES]]
    positions_list.append(''.join([obj_names[i] for i in order]))
len(positions_list)
```

Out[563…
```
12000
```

In [564…
```
pos_state_dict = {'position_state':[val for val in positions_list]}
pos_state_df = pd.DataFrame(data=pos_state_dict, index=df.index)
df = pd.concat([df, pos_state_df], axis=1)
df.head(2)
```

Out[564…

	t	pos	distance_travelled	vel	rot_vel	head_direction	obj_0_pos	obj_0_name	obj_
0	0.05	[0.8927268884855031, 0.505292944589053]	0.003808	[-0.02653824673308769, 0.07139564599352832]	0.524318	[-0.33198065972084784, 0.9432861928234243]	[0.9916465162476297, 0.9887570398068641]	YRGP	[0.294895239852 0.70267500436
1	0.10	[0.8914034882504055, 0.508183352142841]	0.006987	[-0.026468004701951653, 0.057808151075759984]	1.469764	[-0.36041794280043404, 0.932790923255315]	[0.9916465162476297, 0.9887570398068641]	YRGP	[0.294895239852 0.70267500436

2 rows × 307 columns

In [565…
```
len(df), df['position_state'].nunique()/len(df), df['position_state'].nunique()
```

Out[565…
```
(12000, 0.3001666666666667, 3602)
```

In [ ]:

In [566…
```
# help(Ag.plot_trajectory)
```

In [567…
```
df['x_pos'] = df['pos'].explode()[::2]
df['y_pos'] = df['pos'].explode()[1::2]
```

```
/tmp/ipykernel_1360/2856337820.py:1: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` m
any times, which has poor performance.  Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use
`newframe = frame.copy()`
  df['x_pos'] = df['pos'].explode()[::2]
/tmp/ipykernel_1360/2856337820.py:2: PerformanceWarning: DataFrame is highly fragmented.  This is usually the result of calling `frame.insert` m
any times, which has poor performance.  Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use
`newframe = frame.copy()`
  df['y_pos'] = df['pos'].explode()[1::2]
```

In [568…
```
df['y_pos'].max()
```

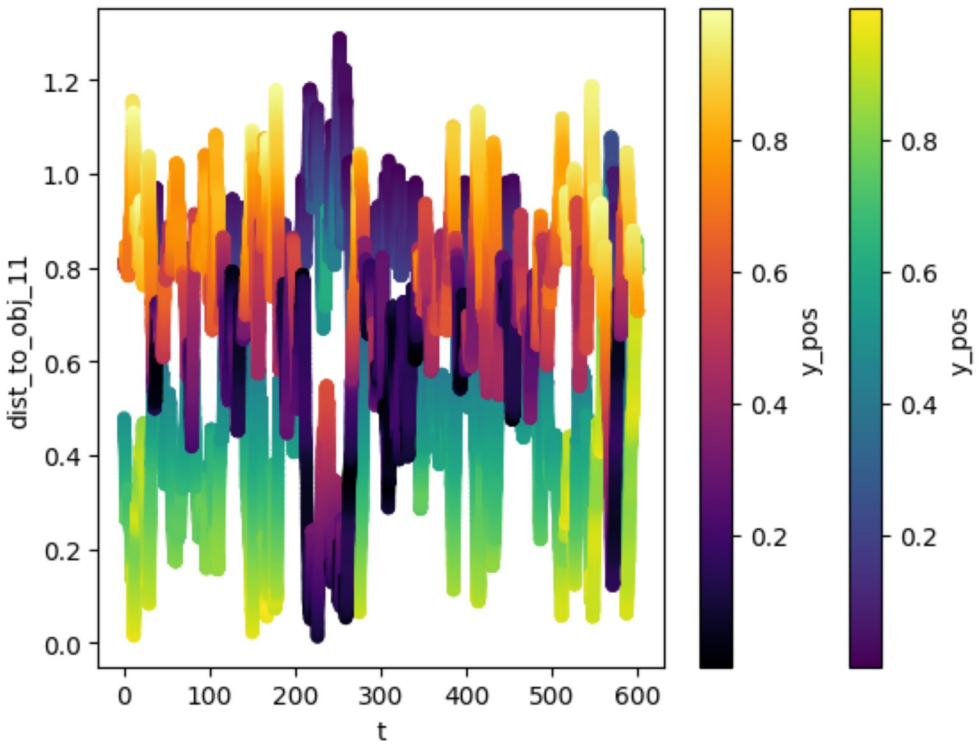
Out[568…
```
0.9991186166282454
```

In [569…
```
# df['pos'].explode()[::2]
```

In [570…
```
df.to_csv(r'/home/mgteus/workspace/navitrans/results_100_objs.csv', index=False)
```

In [571…
```
FIXED_OBJECT = 10
```

In [572…
```
fig, ax = plt.subplots()
df.plot.scatter(x='t', y=f'dist_to_obj_{FIXED_OBJECT}', c='y_pos',ax=ax)
df.plot.scatter(x='t', y=f'dist_to_obj_{FIXED_OBJECT+1}', c='y_pos',ax=ax, cmap='inferno')
plt.show()
```



In [573…
```
# df['last_position_state'] = df['position_state'].shift(1)
```

In [574…
```
# df['change_of_state'] = int(df['position_state'].str == df['last_position_state'].str)
```

In [575…
```
# df['change_of_state'].mean()
```

In [576…
```
pd.set_option('display.max_colwidth', None)
df[['t', 'position_state', 'x_pos', 'y_pos']].iloc[50:55]
```

Out[576…

	t	position_state	x_pos	y_pos
50	2.55	ZCZDUHCYIHASOCIHFCTSWXQLKERA	0.734458	0.739357
51	2.60	ZCZDUHCYOCIHIHASFCTSWXQLKERA	0.731398	0.744061
52	2.65	ZCZDUHCYOCIHIHASFCTSWXQLKERA	0.728271	0.747608
53	2.70	ZCZDUHCYOCIHFCTSIHASWXQLKERA	0.725318	0.750228
54	2.75	ZCZDUHCYOCIHFCTSIHASWXQLKERA	0.721508	0.753281

In [ ]: